

# Traffic Prediction System - Full Documentation

## 1. Import Libraries

```
import streamlit as st
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
```

These libraries are used for building the web app (Streamlit), handling data (Pandas), preprocessing (LabelEncoder), splitting dataset (train\_test\_split), and building ML model (RandomForestClassifier).

## 2. UI Styling

```
st.markdown("""<style>.stApp { background-color: #0E1117; color: white; }</style>""", unsafe_allow_html=True)
```

This line is used to apply custom CSS styling to the Streamlit app to make it visually appealing.

## 3. App Title

```
st.markdown("<h1>■ Traffic Prediction System</h1>", unsafe_allow_html=True)
```

Displays the main heading of the app using HTML formatting.

## 4. Load Dataset

```
df = pd.read_csv("Metro_Interstate_Traffic_Volume.csv")
```

Loads dataset into a dataframe for processing and training.

## 5. Data Preprocessing

```
df['date_time'] = pd.to_datetime(df['date_time'])
df['hour'] = df['date_time'].dt.hour
df['day'] = df['date_time'].dt.dayofweek
```

Converts date to datetime and extracts useful features like hour and day for prediction.

```
df = df.drop(['date_time', 'weather_description'], axis=1)
```

Removes unnecessary columns to simplify the dataset.

## 6. Traffic Level Creation

```
def traffic_level(x):
    if x < 2000:
```

```

        return "Low"
    elif x < 4000:
        return "Medium"
    else:
        return "High"

```

Creates categories from traffic volume for classification.

```
df['traffic_level'] = df['traffic_volume'].apply(traffic_level)
```

Applies the function to convert numeric values into categories.

## 7. Encoding

```

le_weather = LabelEncoder()
le_traffic = LabelEncoder()

```

Encoders convert categorical values into numeric form.

```

df['weather_main'] = le_weather.fit_transform(df['weather_main'])
df['traffic_level'] = le_traffic.fit_transform(df['traffic_level'])

```

Transforms categorical data into numbers for ML model.

## 8. Feature Selection

```

X = df[['temp', 'rain_1h', 'snow_1h', 'clouds_all', 'weather_main', 'hour', 'day']]
y = df['traffic_level']

```

Separates input features and target variable.

## 9. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Splits data into training and testing sets.

## 10. Model Training

```

model = RandomForestClassifier()
model.fit(X_train, y_train)

```

Trains the Random Forest model on the dataset.

## 11. User Input (UI)

```
temp = st.sidebar.number_input("Temperature")
```

Takes input from user for prediction.

## 12. Prediction

```
prediction = model.predict([[temp, rain, snow, clouds, weather_encoded, hour, day]])
```

Predicts traffic level based on user inputs.

## 13. Display Result

Displays the predicted traffic level in a styled format.